

August 2026 - 7

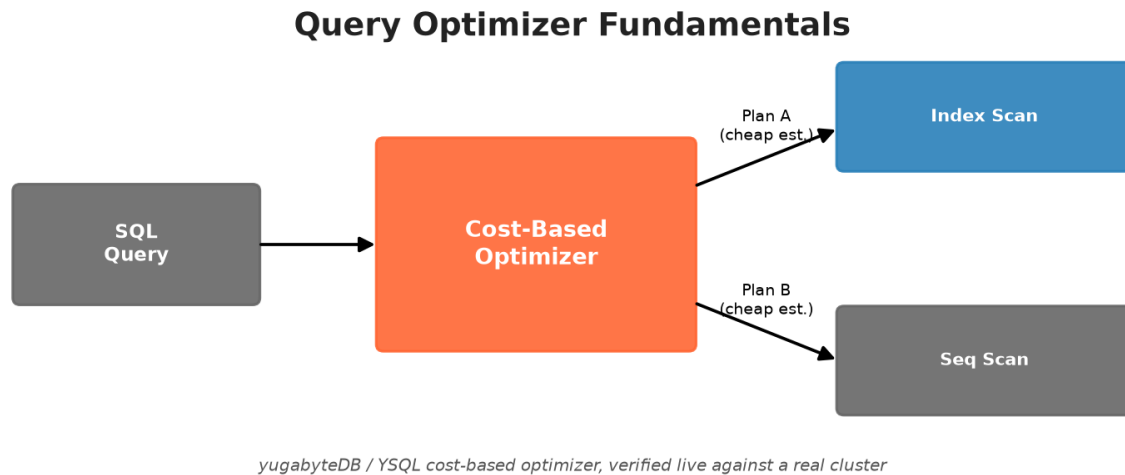


Figure 1: Figure 8-0: Query Optimizer Fundamentals

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

My team keeps hitting query plans we didn’t expect – a query that should obviously use an index does a full table scan instead, or a rewrite that looks slower on paper turns out to be much faster in practice. Is there a mental model for how yugabyteDB actually picks a plan, and some real, worked examples ?

Yes. YSQL’s planner is cost-based, the same lineage as PostgreSQL’s, extended for yugabyteDB’s distributed storage layer. This month we walk through seven real, verified exercises against a live cluster – sharding strategy, pattern-matching indexes, join mechanics, a classic DISTINCT anti-pattern, correlated subqueries, and the difference between SELECT DISTINCT and DISTINCT ON. Every number below came from an actual EXPLAIN (ANALYZE) run, not a guess.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

8.1 Terms and core concepts

8.1.1 Introduction: What Does a Cost-Based Optimizer Actually Do ?

Every SQL engine faces the same problem: a query describes *what* you want, not *how* to get it, and there are usually several different ways to get it. A cost-based optimizer’s job is to estimate, before running anything, roughly how expensive each candidate plan would be – how many rows it would touch, how much I/O, how much sorting – and pick the cheapest-looking one.

YSQL’s optimizer is a direct descendant of PostgreSQL’s, extended to understand yugabyteDB’s distributed storage. That means two things at once are true: everything you already know about reading a PostgreSQL **EXPLAIN** plan mostly transfers directly, and a few new considerations show up that don’t exist on a single-node database – which tablet a row lives on, whether a scan can be pushed down to the storage layer, and whether a join can batch requests across a network hop instead of paying that cost once per row.

[!NOTE] “Cost” in an **EXPLAIN** plan is not a prediction of milliseconds. It is an internal, unitless estimate the planner uses to compare candidate plans against each other. Two plans with costs of 20 and 40,000 are not “2,000x slower” in wall-clock time – they’re just the planner’s best guess at *relative* expense, given its statistics.

This month’s exercises are deliberately small and generic – tables named **t1**, **t2**, columns named **col1**, **col2** – so the pattern being demonstrated stays visible, not buried in a specific business scenario. Every one of them was run for real against a live 8-node cluster; where a number appears below, it came from an actual **EXPLAIN (ANALYZE)** execution.

8.1.2 Sharding Strategy: HASH versus RANGE

The first decision that shapes everything downstream is how a table’s primary key gets distributed across tablets (yugabyteDB’s unit of horizontal partitioning). yugabyteDB supports two sharding strategies per key column: **HASH** and **RANGE** (technicaly, **ASC/DESC**, which yugabyteDB treats as range-sharded).

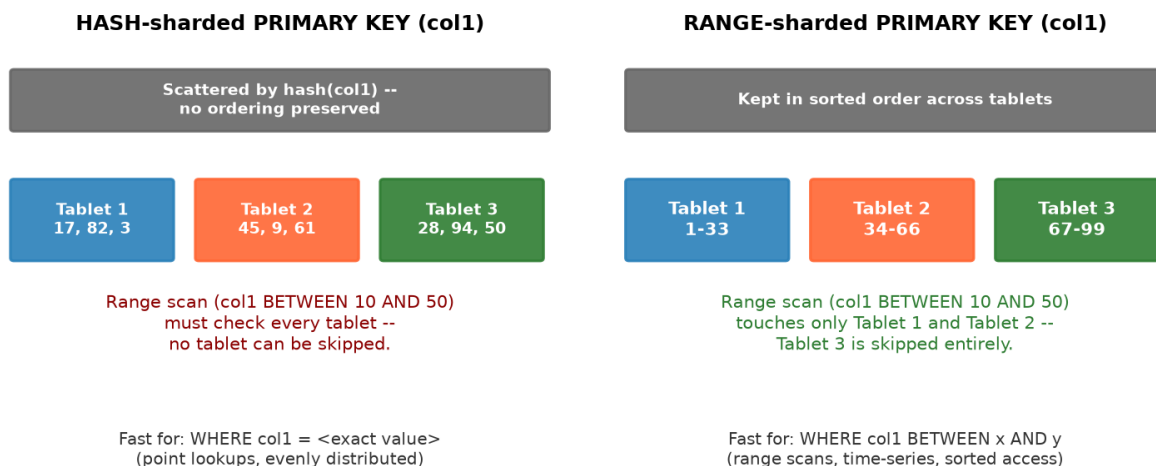


Figure 2: Figure 8-1: HASH vs RANGE sharding

With **HASH** sharding, yugabyteDB applies a hash function to the key and uses the result to pick a tablet. This spreads rows evenly and randomly – excellent for avoiding hot spots on sequential inserts, but it destroys any notion of order. A query like **WHERE col1 BETWEEN 10 AND 50** cannot skip any tablets, because rows 10 and 50 could have landed anywhere.

With **RANGE** sharding, yugabyteDB keeps key values in sorted order, splitting tablets at specific boundary values. The same range query can now skip tablets whose entire key range falls outside [10, 50].

[!NOTE] Neither strategy is “better” in general – they answer different access patterns. A table that’s always queried by exact primary key (a user-lookup table, keyed by UUID) is a natural fit for **HASH**. A table that’s frequently scanned by a range – a time-series table queried by timestamp, an order history table queried by date – is a natural fit for **RANGE**. Exercises 01 and 02 in this month’s lab set demonstrate this directly: a **HASH**-sharded key can serve **WHERE col1 = 10000** instantly, but **WHERE col1 BETWEEN 10000 AND 10100** degenerates into a full scan across every

tablet, because the planner has no way to know which tablets could possibly contain values in that range.

Example 8-1: Declaring each strategy

```
-- HASH sharding (default when unspecified)
CREATE TABLE t1 (col1 INT, PRIMARY KEY (col1 HASH));

-- RANGE sharding
CREATE TABLE t1 (col1 INT, PRIMARY KEY (col1 ASC));
```

8.1.3 Pattern-Matching Indexes: Making LIKE Actually Use an Index

A standard B-tree index on a text column speeds up `col1 LIKE 'prefix%'` for free – the index is sorted, so a prefix search is just a range scan. It does nothing at all for `col1 LIKE '%suffix'` (no anchor to sort by) or `col1 LIKE '%middle%'` (no anchor at either end).

This month’s lab covers three fixes for exactly this gap, from narrowest to most general:

- **The reverse-index trick.** For a *suffix*-only search (`'%suffix'`), index `reverse(col1)` instead of `col1`, and rewrite the query to search `reverse(col1) LIKE reverse('%suffix')` – which becomes a normal, index-friendly *prefix* search on the reversed string.
- **A custom operator.** Wrapping the reverse-index trick behind a purpose-built operator so callers can write natural-looking SQL instead of remembering to reverse both sides by hand.
- **A `pg_trgm` trigram index,** for the fully general case – `'%anything%'`, matching anywhere in the string. Trigram indexes break text into overlapping 3-character sequences and index those, so a substring search becomes a search for rows that share enough trigrams with the pattern.

The trigram approach is the most general, and the exercise’s own numbers show why you’d still reach for the narrower tricks when they apply: a trigram-indexed suffix search against a 5,000-row match set ran in 949.5 ms; a wider match (400,000 rows) against the same index ran in 1,875.9 ms. Both are dramatically faster than the full sequential scan the un-indexed version requires, but a trigram index still has to verify candidates it retrieves (Rows Removed by Index Recheck in the plan) – unlike the reverse-index trick, which returns exact matches with no recheck needed at all.

[!NOTE] `pg_trgm` is a real PostgreSQL extension, and it works on yugabyteDB unmodified: `CREATE EXTENSION pg_trgm;` then `CREATE INDEX ... USING gin (col1 gin_trgm_ops);` (or a B-tree trigram variant, depending on your access pattern). This is a good example of yugabyteDB’s PostgreSQL compatibility paying off directly – an entire, mature indexing strategy arrives for free.

The custom-operator variant is worth walking through in full, because the *first* attempt at it fails in an instructive way. The goal: let a caller write `col1 ~~^ 'GE AK'` (natural, unreversed search text) instead of manually reversing both the column and the literal by hand every time.

Example 8-1a: Attempt 1 – looks reasonable, verified NOT to use the index

```
CREATE FUNCTION suffix_match_naive(text, text) RETURNS boolean AS $func$
    SELECT reverse($1) LIKE (reverse($2) || '%')
$func$ LANGUAGE sql IMMUTABLE;

CREATE OPERATOR ~~^ (
    LEFTARG = text, RIGHTARG = text, PROCEDURE = suffix_match_naive
);

EXPLAIN SELECT * FROM t2 WHERE col1 ~~^ 'GE AK';
-- Seq Scan on t2
--   Storage Filter: (reverse(col1) ~~ 'KA EG%':::text)
```

Even though the planner inlines the SQL-language function (you can see the expanded `reverse(col1) ~~ ...` form directly in the plan, proof the inlining worked), it still computes `reverse(col1)` fresh, on the fly, for every row – not the same expression as a stored, indexed `col1_rev` column. Matching a stored generated column to an index requires the query to reference *that column by name*; recomputing an equal *value* via a different expression doesn't count, no matter how obviously equivalent the two expressions are to a human reading them. The planner has no general mechanism for recognizing that kind of semantic equivalence – it matches expressions, not meanings.

Example 8-1b: Attempt 2 – references the real, indexed column directly; verified to use the index

```
CREATE FUNCTION reverse_suffix_match(text, text) RETURNS boolean AS $func$
    SELECT $1 LIKE (reverse($2) || '%')
$func$ LANGUAGE sql IMMUTABLE;
```

```
CREATE OPERATOR ~~^ (
    LEFTARG = text, RIGHTARG = text, PROCEDURE = reverse_suffix_match
);
```

```
EXPLAIN (ANALYZE)
SELECT * FROM t2 WHERE col1_rev ~~^ 'GE AK';
-- Index Scan using t2_col1_rev_idx on t2 (actual time=22.209..70.513 rows=5000 loops=1)
--   Index Cond: ((col1_rev >= 'KA EG'::text) AND (col1_rev < 'KA EH'::text))
--   Storage Index Filter: (col1_rev ~~ 'KA EG'::text)
-- Execution Time: 74.450 ms
```

The only difference between the two attempts: the operator's left argument is the *stored, indexed* `col1_rev` column itself, not a freshly-recomputed `reverse(col1)`. The operator only hides the “reverse the search literal and append %” bookkeeping – the caller still has to know to query `col1_rev` instead of `col1` (nothing in plain SQL can hide that without a planner hook, out of reach here), but the error-prone “reverse my search string by hand” step is gone, and the plan is a genuine *Index Scan*, confirmed at 74.5 ms.

8.1.4 Join Mechanics: Why a LEFT JOIN's Preserved Side Drives the Loop

A *LEFT JOIN* has a “preserved” side (every row from the left table appears in the result, matched or not) and a side being probed. Against a 100,000-row `t1` (preserved) and a 10,000-row `t2` (90% of `t1` has no match at all, a genuine outer join, not one that could just as well have been written as an inner join), the planner's default choice is a *Hash Left Join*:

```
Hash Left Join (actual time=27.894..145.244 rows=100000 loops=1)
  Hash Cond: (t1.col1 = t2.col1)
    -> Seq Scan on t1 (actual time=3.513..84.543 rows=100000 loops=1)
    -> Hash (actual time=24.310..24.311 rows=10000 loops=1)
      -> Seq Scan on t2 (actual time=1.677..21.353 rows=10000 loops=1)
Execution Time: 167.949 ms
```

The smaller table (`t2`) gets built into the in-memory hash table first, and the larger preserved table (`t1`) streams through as the probe – “build the smaller side, probe with the larger” already happens by default, no hint required.

The more interesting question: can a hint force a *Nested Loop* plan to drive from `t2` instead of `t1` – i.e., flip which table is the outer loop? Tested two ways, live: first by disabling hash and merge joins (`SET enable_hashjoin = off; SET enable_mergejoin = off;`), which produced a *YB Batched Nested Loop Left Join* with `t1` as the driving/outer side and `t2` probed via an *Index Scan* – and second, adding an explicit `pg_hint_plan Leading((t2 t1))` hint on top, requesting `t2` first. The second attempt produced the byte-for-byte identical plan. The hint had no alternative plan to select from.

The reason is semantic, not a cost-model limitation: a *Nested Loop* implementing a *LEFT JOIN* must iterate

every row of the preserved side exactly once, to correctly NULL-extend rows with no match – and that only works if the preserved side is the outer, driving loop. Driving from `t2` instead would only ever visit `t2`'s rows, never surfacing `t1`'s 90,000 unmatched rows at all – not an alternate strategy for the *same* query, a different (wrong) answer to a different question. This is exactly why no hint can force it: `pg_hint_plan`'s `Leading()` only reorders among plans the optimizer would already consider *valid* – it constrains the search space, it doesn't override join semantics.

8.1.5 The SELECT DISTINCT Anti-Pattern – and Fixing It Structurally

A very common shape: join two tables, and because the join fans out (one entity, many matching child rows), reach for `SELECT DISTINCT` to collapse the duplicates back down. It works, but it means paying a sort-or-hash de-duplication cost on *every single query*, forever, for a fact that doesn't actually change very often (which entities qualify).

This month's lab rebuilds that pattern three ways:

1. The naive version: join, then `DISTINCT`. Correct, but the de-duplication cost is paid on every read.
2. Precompute the qualifying set *once*, into its own table (`t3`, one row per qualifying entity, enforced by its own primary key) – then future queries join against `t3` directly and never need `DISTINCT` at all, because `t3` structurally cannot contain duplicates.
3. Keep `t3` in sync automatically with a trigger on the source table, handling the genuinely tricky part correctly: adding a qualifying row is easy (insert, `ON CONFLICT DO NOTHING`), but *removing* one requires checking whether the entity still qualifies through some *other* row before deleting it.

The trade-off this demonstrates: the cost of `DISTINCT` doesn't disappear, it moves – from every future read, to one maintenance step (the trigger) paid only when the underlying data actually changes. For a read-heavy table, that's usually a very good trade.

The trigger's actual logic is short, and worth reading in full, because the “removing a row” direction is the part that's easy to get subtly wrong:

Example 8-1c: Keeping the qualifying-set table in sync automatically

```
CREATE OR REPLACE FUNCTION t2_maintain_t3() RETURNS trigger AS $func$
BEGIN
    IF TG_OP = 'INSERT' THEN
        IF NEW.status = 'active' THEN
            INSERT INTO t3 (col1) VALUES (NEW.col1) ON CONFLICT DO NOTHING;
        END IF;
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.status = 'active' AND OLD.status IS DISTINCT FROM 'active' THEN
            INSERT INTO t3 (col1) VALUES (NEW.col1) ON CONFLICT DO NOTHING;
        ELSIF OLD.status = 'active' AND NEW.status IS DISTINCT FROM 'active' THEN
            IF NOT EXISTS (SELECT 1 FROM t2 WHERE col1 = OLD.col1 AND status = 'active') THEN
                DELETE FROM t3 WHERE col1 = OLD.col1;
            END IF;
        END IF;
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        IF OLD.status = 'active' THEN
            IF NOT EXISTS (SELECT 1 FROM t2 WHERE col1 = OLD.col1 AND status = 'active') THEN
                DELETE FROM t3 WHERE col1 = OLD.col1;
            END IF;
        END IF;
    END IF;
END IF;
```

```

    RETURN OLD;
END IF;
END;
$func$ LANGUAGE plpgsql;

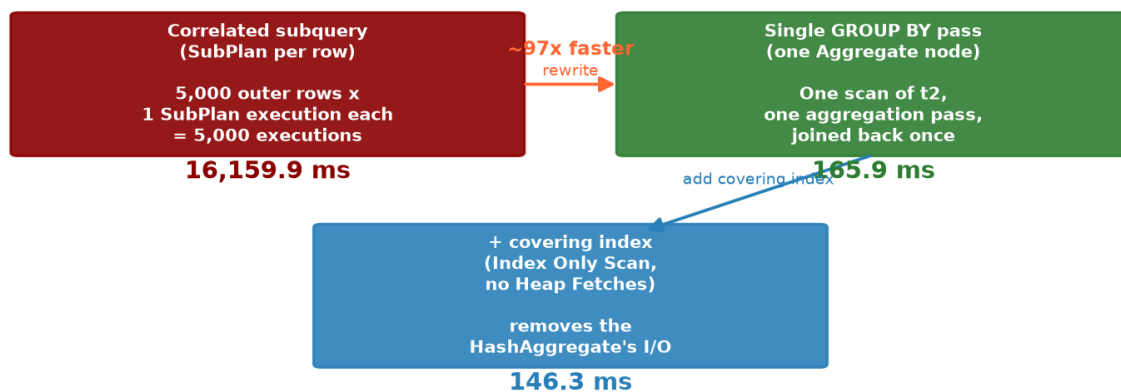
CREATE TRIGGER t2_t3_sync
AFTER INSERT OR UPDATE OR DELETE ON t2
FOR EACH ROW EXECUTE FUNCTION t2_maintain_t3();

```

Adding a qualifying row is the easy direction: a new active event means the entity belongs in **t3**, so just insert it, **ON CONFLICT DO NOTHING** (idempotent no matter how many active events that entity ends up with). Removing one needs the extra check: an entity's *last* active event going away is genuinely different from *an* active event going away – the **NOT EXISTS** check confirms no other active event remains for that entity before deleting it from **t3**. Skipping that check is the classic bug this pattern invites: deleting from **t3** the moment *any* active event disappears, even while other active events for the same entity still exist.

8.1.6 Correlated Subqueries versus a Single GROUP BY Pass

A correlated subquery – one that references a column from the outer query – gets re-planned and re-executed once *per outer row*. For a small number of outer rows this is invisible. For thousands of outer rows, it becomes a very literal, very measurable cost.



All three numbers measured live via EXPLAIN (ANALYZE) against the same 5,000-row query.

Figure 3: Figure 8-2: Correlated subquery vs. GROUP BY rewrite, real numbers

This month's exercise makes the cost visible rather than theoretical. Against a 5,000-row outer query, the correlated-subquery version executed in **16,159.9 ms**. Rewriting it as a single **GROUP BY** pass – computing the aggregate once for every group, then joining the result back – brought that down to **165.9 ms**, roughly a 97x improvement, simply by turning 5,000 small executions into one. Adding a covering index on top (so the aggregation could run as an **Index Only Scan**, with zero **Heap Fetches**) brought it down further to **146.3 ms** – a smaller, but real, additional gain from removing the aggregate's remaining I/O.

[!NOTE] The general shape to watch for: if a subquery in your **SELECT** list or **WHERE** clause references the outer query, and the outer query returns more than a handful of rows, ask whether the subquery's logic could instead be expressed as a **GROUP BY** (or a window function) computed

once, then joined. It very often can be, and the planner's SubPlan line in EXPLAIN – with a loops=N count matching your outer row count – is the tell.

8.1.7 SELECT DISTINCT versus DISTINCT ON – a Different Job, Not a Faster One

It's tempting to assume PostgreSQL's DISTINCT ON (expr) – which keeps exactly one row per group, chosen by an ORDER BY – is simply a faster version of plain DISTINCT. Measured directly, it is not, and understanding why is more useful than the numbers themselves.

Plain DISTINCT on (col1, status) can only remove rows that are *identical* across every selected column. Where every entity has both an active and an inactive event, (entity, 'active') and (entity, 'inactive') are two different rows to DISTINCT – not duplicates – so it correctly returns roughly twice as many rows as there are entities (**200,000 rows**, confirmed live, against 100,000 entities).

DISTINCT ON (col1) solves a genuinely different problem: exactly one row per col1, deterministically chosen by the trailing ORDER BY. Measured against the same data, it correctly returned **100,000 rows** – but it was not faster (2,102.3 ms versus the plain version's 1,957.9 ms). The reason is structural, not incidental: plain DISTINCT can use a HashAggregate, collapsing duplicates without needing the input in any particular order first. DISTINCT ON has no such option – PostgreSQL/YSQL implements it only as sort-then-take-first-per-group, so it must fully sort the entire input (in this case, all 1,000,000 underlying rows) before it can pick winners, even though the final answer is ten times smaller than what it had to sort to get there.

[!NOTE] The fix, where DISTINCT ON performance matters, is the same fix that always helps a sort-dependent plan: a supporting index in the same order the query needs, (col1, id DESC) in this case. With that index in place, the planner can walk it directly and take the first row per group via an Index Scan, with no Sort node at all.

8.1.8 A Real Customer Case: Separating Join Order From Index Column Order

This month's final exercise is anonymized from a real customer case study – three tables, t1 (a near-constant “location code” plus a truly unique column), t2 (a weak join key back to t1, plus a truly selective, 2%-matching column that also joins to t3), and t3 (a clean, unique lookup table that was never part of the problem). The original indexes led with the near-constant, weakly-selective column – a very common, very natural-looking mistake, since that column is also the join key and “the thing you filter by” in a lot of related queries.

Three variants of the same query were run against this same data, isolating two different effects that the original case study's own numbers had bundled together into one:

- **t1 -> t2 -> t3, forced via pg_hint_plan into the original, sub-optimal join order and index access path** – deliberately reproducing what an older, less capable optimizer picked on its own: **~385-395 ms**.
- **The same query and data, no hints, unmodified original indexes** – whatever YSQL's cost-based optimizer chooses naturally: **~37-43 ms**, roughly a 9-10x improvement from join order and access path alone.
- **The case study's actual fix applied** – secondary indexes re-ordered to lead with the truly selective column instead of the near-constant one, still no hints: **~10-12 ms**, a further ~3-4x improvement.

-- The fixed variant's plan (t5 = t2's data, reindexed col3-first):

Nested Loop (actual time=7.587..12.984 rows=1900 loops=1)

-> Index Scan using t3_pkey on t3 (actual time=0.268..0.269 rows=1 loops=1)

Index Cond: (col1 = 'FLT-00025'::text)

-> YB Batched Nested Loop Join (actual time=7.315..12.277 rows=1900 loops=1)

-> Index Only Scan using idx_t5_col3_col2_col1 on t5 (rows=1900 loops=1)

Index Cond: (col3 = 'FLT-00025'::text)

Heap Fetches: 0

-> Index Scan using t4_pkey on t4 (rows=700 loops=2)

Execution Time: 16.132 ms

The two effects are genuinely independent, and worth keeping separate in your own head when diagnosing a slow query: the **~9-10x** gap between the forced-bad-plan and the natural-plan runs is entirely a *join order and access path* effect – something only a cost-based optimizer with real statistics can get right on its own, and something no amount of re-indexing fixes if the optimizer is prevented from choosing freely. The further **~3-4x** gap is entirely an *index column order* effect – additive on top of good join-order decisions, not a substitute for them. A slow query in production is very often *both* problems stacked, and treating it as one undifferentiated “make it faster” task tends to under-fix it; separating the two, as this exercise does explicitly, tells you which lever to pull first.

8.2 Complete the following

In this section, we reproduce the correlated-subquery-versus-GROUP BY measurement from section 8.1.6 end to end, on your own cluster, so the numbers above are not something you have to take on faith.

We will: create a small two-table dataset shaped to trigger the correlated-subquery pattern, run and time the naive version, rewrite it as a GROUP BY pass and re-time it, then add a covering index and time it a third time. By the end, you will have reproduced (approximately) the 16,159.9 ms → 165.9 ms → 146.3 ms progression from section 8.1.6 on your own hardware.

8.2.1 Prerequisites

A running yugabyteDB cluster. A single-node cluster is sufficient for this exercise; a three-node cluster with replication factor 3 more closely matches production behavior.

Example 8-2: For a single-node development cluster:

```
./bin/yugabyted start --advertise_address 127.0.0.1
```

8.2.2 Step 1 – Create the dataset

Example 8-3: Two tables, sized to make the correlated-subquery cost visible

```
CREATE TABLE t1 (  
    col1 INT,  
    col2 TEXT,  
    PRIMARY KEY (col1 HASH)  
);  
  
CREATE TABLE t2 (  
    id BIGINT,  
    col1 INT,  
    col2 INT,  
    PRIMARY KEY (id HASH)  
);  
  
CREATE INDEX idx_t2_col2 ON t2 (col2 HASH);  
  
INSERT INTO t1 (col1, col2)  
SELECT g, (ARRAY['East', 'West', 'North', 'South'])[1 + (g % 4)]  
FROM generate_series(1, 5000) AS g;  
  
INSERT INTO t2 (id, col1, col2)  
SELECT g, (g % 100000) + 1, (g % 20) + 1  
FROM generate_series(1, 100000) AS g;  
  
ANALYZE t1;  
ANALYZE t2;
```

8.2.3 Step 2 – Run and time the correlated-subquery version

Example 8-4: The naive version – one SubPlan execution per outer row

```
EXPLAIN (ANALYZE)
SELECT
    t1.col1,
    (SELECT count(*) FROM t2 WHERE t2.col2 = t1.col1) AS matching_count
FROM t1
WHERE t1.col2 = 'West';
```

Look for the SubPlan node in the output, and note its `loops= count` – it should match the number of `t1` rows matching `col2 = 'West'` (roughly 1,250 of the 5,000, given four evenly-split regions). Note the **Execution Time** at the bottom.

8.2.4 Step 3 – Rewrite as a single GROUP BY pass

Example 8-5: One aggregation pass over t2, joined back once

```
EXPLAIN (ANALYZE)
SELECT
    t1.col1,
    COALESCE(t2_counts.matching_count, 0) AS matching_count
FROM t1
LEFT JOIN (
    SELECT col2, count(*) AS matching_count
    FROM t2
    GROUP BY col2
) AS t2_counts
ON t2_counts.col2 = t1.col1
WHERE t1.col2 = 'West';
```

Compare **Execution Time** against Step 2. On this project’s own cluster, this step alone produced roughly a 97x improvement (16,159.9 ms → 165.9 ms) on a similarly-shaped dataset – your exact numbers will depend on your cluster’s sizing, but the *direction and rough magnitude* of the improvement should be unmistakable.

8.2.5 Step 4 – Add a covering index

Example 8-6: An index that lets the GROUP BY run as an Index Only Scan

```
CREATE INDEX idx_t2_col2_covering ON t2 (col2) INCLUDE (id);
ANALYZE t2;
```

Re-run the query from Step 3. Confirm in the plan that the scan feeding the **GroupAggregate** is now an **Index Only Scan with Heap Fetches: 0** – meaning yugabyteDB satisfied the query entirely from the index, without a trip to the underlying table at all.

`INCLUDE (id)` is doing specific, deliberate work here: it stores `id` alongside the index’s key column (`col2`) without making `id` part of the index’s *sort order*. The **GROUP BY** only needs to group by `col2`; it doesn’t care what order rows within a group arrive in, so there’s no reason to pay for `id` to be sorted too. `INCLUDE` gets you the “no heap trip needed” benefit of a covering index without inflating the index’s actual key – a distinction worth knowing, since a naive `CREATE INDEX idx ON t2 (col2, id)` would also technically “cover” this query, but at the cost of a wider, more expensive-to-maintain index for no query benefit.

8.2.6 Step 5 – Confirm the result set didn’t change

A rewrite that returns a different (even if faster) answer is not a successful rewrite. Before trusting any of the numbers above, confirm all three query forms agree on the actual data:

Example 8-7: Cross-checking correctness, not just speed

```
SELECT count(*) FROM (
  SELECT col1, (SELECT count(*) FROM t2 WHERE t2.col2 = t1.col1) AS matching_count
  FROM t1 WHERE t1.col2 = 'West'
) AS naive
FULL OUTER JOIN (
  SELECT t1.col1, COALESCE(t2_counts.matching_count, 0) AS matching_count
  FROM t1
  LEFT JOIN (SELECT col2, count(*) AS matching_count FROM t2 GROUP BY col2) AS t2_counts
  ON t2_counts.col2 = t1.col1
  WHERE t1.col2 = 'West'
) AS rewritten
ON naive.col1 = rewritten.col1 AND naive.matching_count = rewritten.matching_count
WHERE naive.col1 IS NULL OR rewritten.col1 IS NULL;
```

This should return 0 – no row present in one result set and not the other. It’s a small step, and easy to skip when a rewrite “obviously” preserves semantics, but the correlated-subquery-to-`GROUP BY` rewrite specifically has a real failure mode worth guarding against: if the outer query’s join to the aggregated subquery were accidentally an `INNER JOIN` instead of a `LEFT JOIN`, entities with a `matching_count` of zero would silently disappear from the result entirely, rather than showing 0 as the naive version does. The query above catches exactly that class of mistake.

Section summary In this section, we built a small two-table dataset specifically shaped to trigger the correlated-subquery cost from section 8.1.6, then reproduced its fix in two stages: first rewriting the correlated subquery as a single `GROUP BY` pass (the large win), then adding a covering index on top (a smaller, additional win from eliminating the aggregate’s remaining I/O). We closed by cross-checking that the rewrite preserves the exact same result set as the original – a step worth never skipping, since the specific rewrite shown here has a real, easy-to-introduce failure mode (an accidental `INNER JOIN` silently dropping zero-count rows) that only a correctness check, not a faster `EXPLAIN` plan, will catch. The exact millisecond numbers will vary by cluster, but the shape of the improvement – one big structural win, followed by a smaller I/O-elimination win – should reproduce reliably.

8.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- Eight verified query-optimizer exercises against a live yugabyteDB cluster: `HASH` versus `RANGE` sharding and how each serves (or fails to serve) range scans; three pattern-matching index strategies for `LIKE` queries that a plain B-tree can’t otherwise help with, including a custom operator whose first implementation attempt looked reasonable and verified as *not* using the index; why a `LEFT JOIN`’s preserved side always drives the loop, with no hint able to change it; the `SELECT DISTINCT` anti-pattern and its structural fix via a maintained, trigger-synced table; a correlated subquery rewritten into a single `GROUP BY` pass for a measured ~97x improvement; why `DISTINCT ON` is a different job from `DISTINCT`, not a faster version of it; and a real, anonymized customer case separating a ~9-10x join-order effect from an independent, additive ~3-4x index-column-order effect.
- A hands-on, step-by-step reproduction of the correlated-subquery-to-`GROUP BY` rewrite, including the covering-index follow-up, so the numbers in this article can be verified independently rather than taken on faith.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-08%20-%20Query%20Optimizer%20Fundamentals%20Query%20optimizer%20fundamentals.pdf>